

Ejercicios: recursión y listas
Unidad 0: Herramientas y preliminares

Eduardo Acuña Yeomans

Lenguajes de Programación
Semestre 2026-2

Lic. en Ciencias de la Computación
Universidad de Sonora

Sesión 2 · Practica: S0-1

Esta hoja no se entrega y no se califica. Es práctica, y se verifica sola: cada ejercicio trae las llamadas que tienes que probar y lo que Racket debe contestar. Si te da otra cosa, algo está mal y vale la pena averiguar qué.

Trabaja en un archivo `.rkt` que empiece con `#lang racket`, presiona Run, y teclea las llamadas en el REPL.

Sugerencia: la receta, por si se te va

Tres preguntas, siempre en este orden: ¿cuál es el caso en el que ya sé la respuesta sin calcular nada? ¿Cómo hago el problema más chico? ¿Cómo combino la respuesta del problema chico con lo mío?

Calentamiento: recursión sobre números

1. cuenta-atras Recibe un número n y devuelve la lista de n hasta 1. Con $n = 0$, la lista vacía.

```
(cuenta-atras 5) = '(5 4 3 2 1)
(cuenta-atras 1) = '(1)
(cuenta-atras 0) = '()
```

2. suma-hasta Suma todos los números de 1 a n .

```
(suma-hasta 5) = 15
(suma-hasta 0) = 0
```

3. potencia Eleva b a la e . Sin usar `expt`, claro.

```
(potencia 3 4) = 81
(potencia 5 0) = 1
```

Listas

Los cuatro que siguen ya existen en Racket con otros nombres. Escríbelos tú para ver qué hay adentro.

4. **longitud** Cuántos elementos tiene una lista.

```
(longitud '(a b c)) = 3
(longitud '())      = 0
```

5. **concatenar** Pega dos listas y devuelve una nueva.

```
(concatenar '(1 2) '(3 4 5)) = '(1 2 3 4 5)
(concatenar '() '(1))        = '(1)
```

Sugerencia: el caso base ya no es un número

En **longitud** el caso base era 0. Aquí, cuando la primera lista se acaba, ¿qué queda por devolver?

6. **mapea** Aplica una función a cada elemento. Recibe la función como primer argumento.

Para probarlo define antes (**define** (**cuadrado** x) (* x x)) y (**define** (**doble** x) (* x 2)).

```
(mapea cuadrado '(1 2 3 4)) = '(1 4 9 16)
(mapea doble '(10 20))      = '(20 40)
```

7. **filtra** Se queda solo con los elementos que pasan una prueba. Recibe la prueba como primer argumento.

even? ya viene en Racket. Define tú (**define** (**positivo?** n) (> n 0)).

```
(filtra even? '(1 2 3 4 5 6)) = '(2 4 6)
(filtra positivo? '(-2 5 0 7 -1)) = '(5 7)
```

Sugerencia: aquí hay tres casos, no dos

La lista está vacía, el primer elemento pasa la prueba, o el primer elemento no la pasa. Con tres casos, **cond** se lee mejor que **if** anidados.

Combinar lo anterior

8. **invierte** Devuelve la lista al revés. Usa **concatenar** en la solución.

```
(invierte '(1 2 3 4)) = '(4 3 2 1)
(invierte '(a))       = '(a)
```

9. suma-lista Suma todos los números de una lista.

```
(suma-lista '(10 20 30)) = 60  
(suma-lista '()) = 0
```

10. Sin escribir código ¿Qué devuelve (suma-lista (mapea cuadrado (cuenta-atras 3)))?
Contéstalo en papel primero, y después compruébalo.

Soluciones

No las mires hasta haber intentado cada ejercicio. Si tu versión es distinta de la de aquí pero da las mismas salidas, está bien: casi nunca hay una sola forma.

Clave de Respuesta: calentamiento

```
(define (cuenta-atras n)
  (if (= n 0)
      '()
      (cons n (cuenta-atras (- n 1)))))

(define (suma-hasta n)
  (if (= n 0)
      0
      (+ n (suma-hasta (- n 1)))))

(define (potencia b e)
  (if (= e 0)
      1
      (* b (potencia b (- e 1)))))
```

Clave de Respuesta: listas

```
(define (longitud lst)
  (if (null? lst)
      0
      (+ 1 (longitud (cdr lst)))))

(define (concatenar l1 l2)
  (if (null? l1)
      l2
      (cons (car l1) (concatenar (cdr l1) l2))))

(define (mapea f lst)
  (if (null? lst)
      '()
      (cons (f (car lst)) (mapea f (cdr lst)))))

(define (filtra p lst)
  (cond [(null? lst) '()]
        [(p (car lst)) (cons (car lst) (filtra p (cdr lst)))]
        [else (filtra p (cdr lst))]))
```

Clave de Respuesta: combinar lo anterior

```
(define (invierte lst)
  (if (null? lst)
      '()
      (concatenar (invierte (cdr lst)) (list (car lst)))))

(define (suma-lista lst)
  (if (null? lst)
      0
      (+ (car lst) (suma-lista (cdr lst)))))
```

En `invierte`, fíjate en `(list (car lst))`: hace falta una *lista* de un elemento, porque `concatenar` pega dos listas. Con `(car lst)` suelto no funcionaría.

Ejercicio 10. `(cuenta-atras 3)` da `'(3 2 1)`. Elevar al cuadrado da `'(9 4 1)`. Sumar da `14`.